

問題4-6: 損失 + 波長分散の多中継伝送とデジタル分散補償後の BER

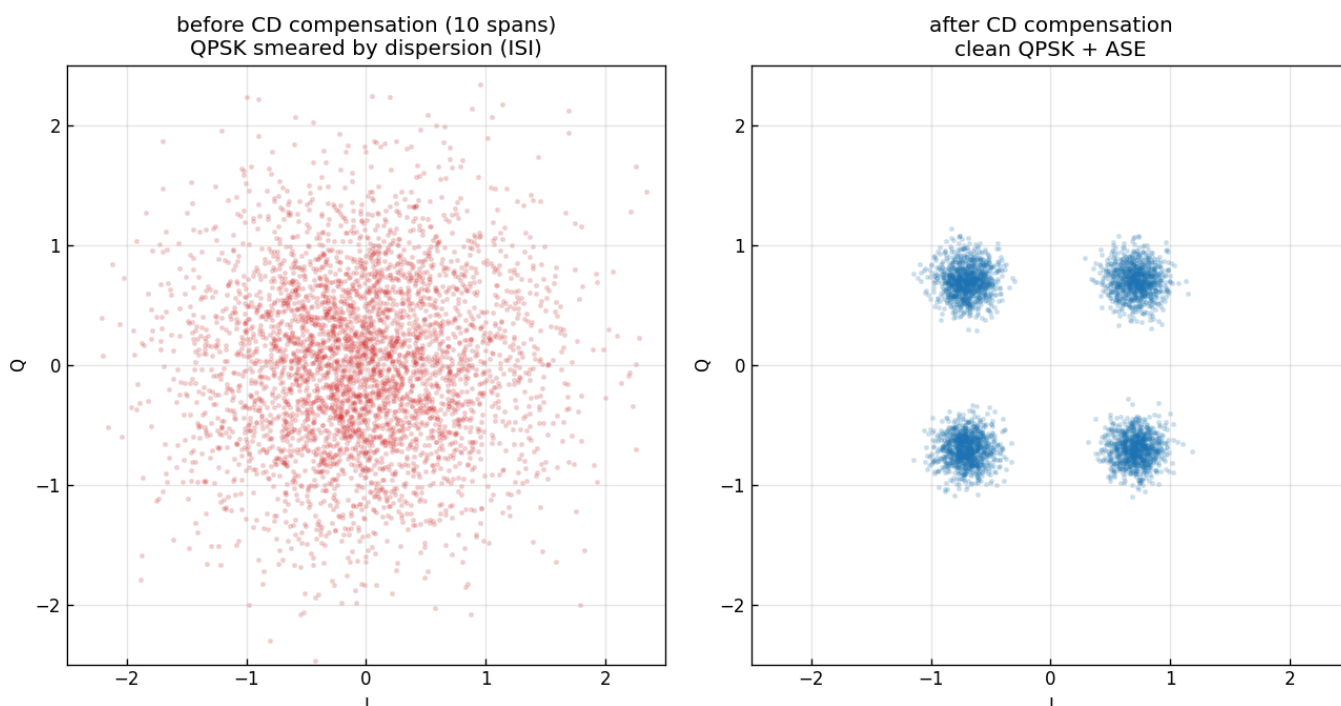
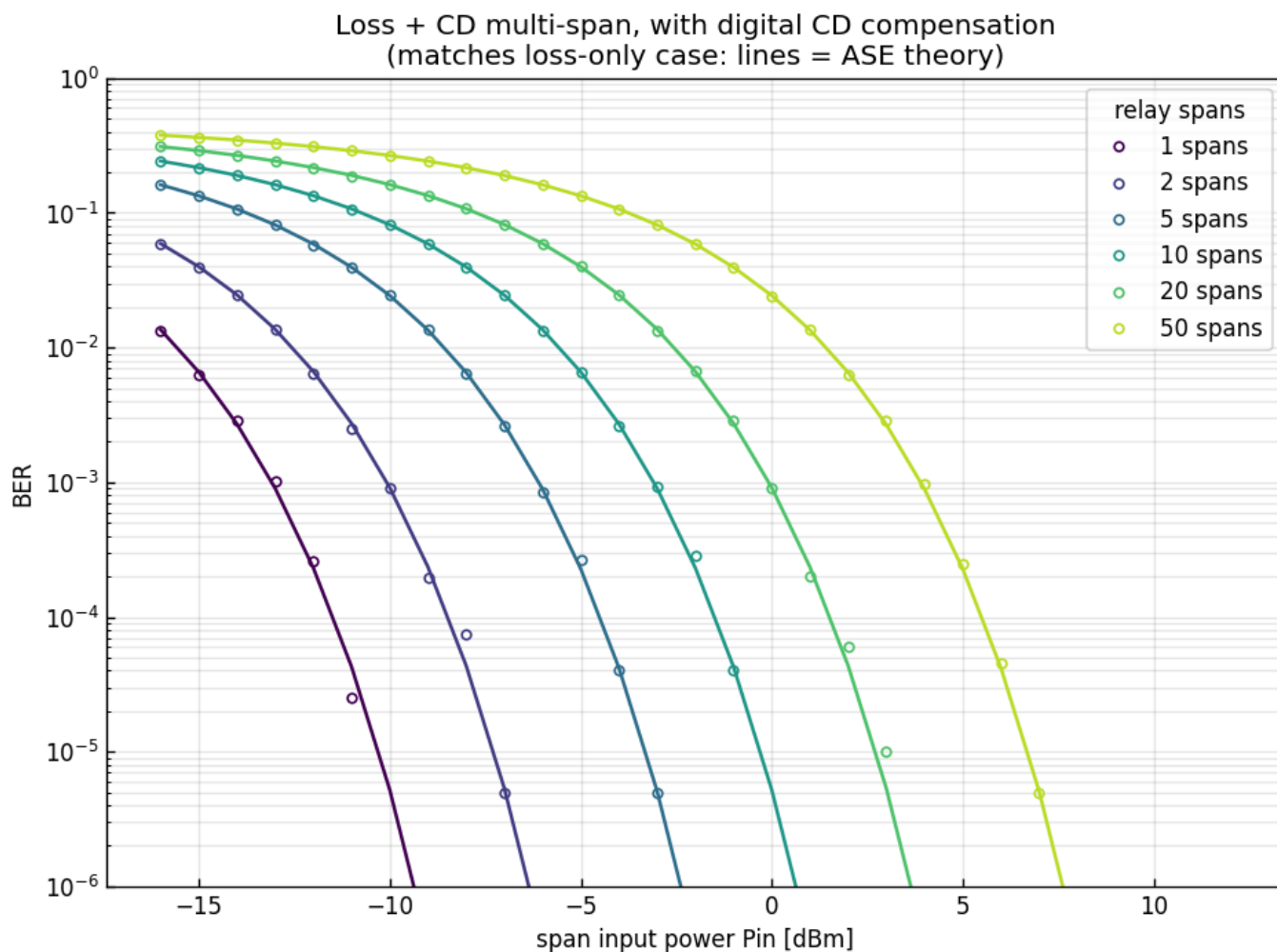
問題

問4-5(損失のみの多中継)に **波長分散**を加える。伝送路は損失係数 0.3 dB/km、二次分散 $\beta_2 = -22 \text{ ps}^2/\text{km}$ のファイバを 100 km ごとに区切ったスパンの連結とする。各スパンの 末尾に EDFA(NF=4 dB)を置き、スパン損失 30 dB をちょうど打ち消す利得を与える(各スパンの入力 パワーは 同一)。1, 2, 5, 10, 20, 50 スパン中継した後にコヒーレント受信し、**蓄積した総波長分散を 受信側でまとめて完全補償**してから QPSK を復調する。BER の入力パワー P_{in} 依存性を求める(レーザ位相雑音は無視)。

実行結果(出力)

$\beta_2 = -22.0 \text{ ps}^2/\text{km}$, スパン 100 km, 損失/利得 30 dB, NF 4.0 dB

1 spans (100 km):	受信感度(BER=1e-3) $\approx$	-13.0 dBm
2 spans (200 km):	受信感度(BER=1e-3) $\approx$	-10.1 dBm
5 spans (500 km):	受信感度(BER=1e-3) $\approx$	-6.1 dBm
10 spans (1000 km):	受信感度(BER=1e-3) $\approx$	-3.1 dBm
20 spans (2000 km):	受信感度(BER=1e-3) $\approx$	-0.1 dBm
50 spans (5000 km):	受信感度(BER=1e-3) $\approx$	4.0 dBm



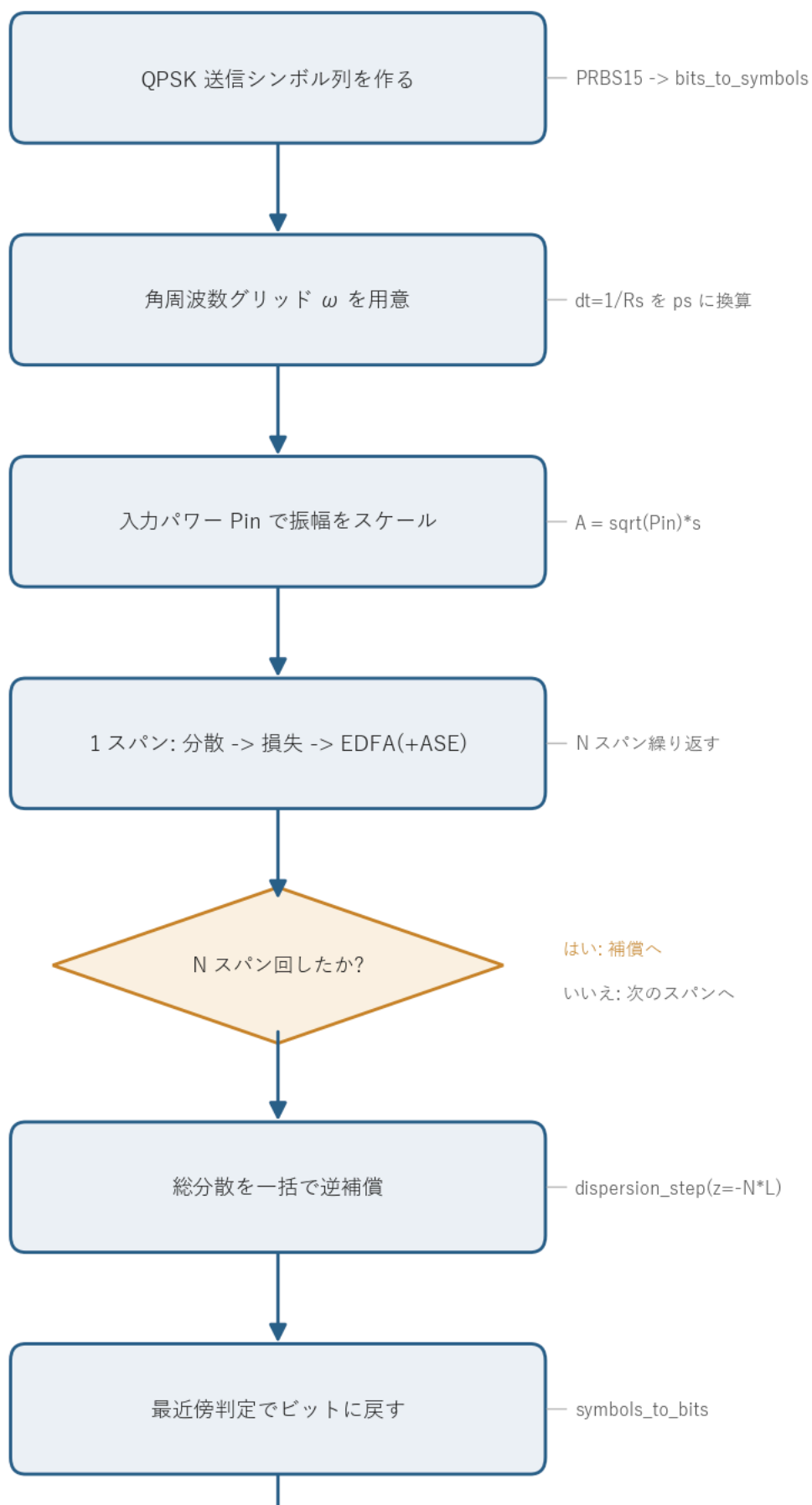
図の読み方 - 上: 分散を補償した後の BER 曲線。スパン数を変えても、各曲線は間4-5(損失のみ)と完全に重なる。波長分散を足してもペナルティはゼロ。スパンが増えるほど右へ寄り、必要な入力

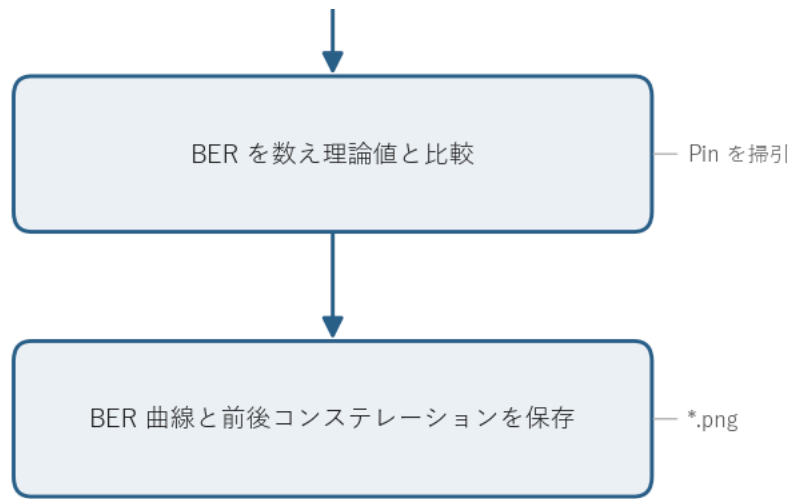
パワーが上がる。 - **下(左)**: 10 スパン伝送後、補償前のコンステレーション。QPSK の 4 点が分散による符号間干渉 (ISI) で雲状に崩れている。 - **下(右)**: 同じ信号を分散補償した後。きれいな QPSK の 4 点に戻り、残るのは ASE による丸い広がりだけ。

0. 処理の流れ

`solution.py` を実行すると、次の順で処理が進む。

損失+分散の多中継と分散補償の処理フロー





QPSK のシンボル列を作って入力パワーでスケールし、スパンごとに「分散 → 損失 → EDFA」を繰り返す。全スパンを通したら、蓄積した総分散を受信側で 1 回の逆位相フィルタで打ち消し、最近傍判定でビットに戻して BER を数える。これを入力パワー P_{in} とスパン数 N の組み合わせで掃引するのが本体のループになる。分岐はスパン繰り返しの終了判定だけで、伝搬から判定までは一本道でつながっている。

1. 波長分散は線形で可逆

波長分散は、周波数ごとに違う位相を与えるフィルタとして書ける。スペクトルの各成分 ω に対し、距離 z を進んだときの伝達関数は

$$H_{CD}(\omega) = \exp\left(j\frac{\beta_2}{2}\omega^2 z\right), \quad |H_{CD}(\omega)| = 1$$

となる。振幅 $|H_{CD}|$ がすべての周波数で 1 なので、これは **全域通過(オールパス)フィルタ** である。振幅を一切変えず、位相だけを ω^2 に比例して回す。位相が周波数で変わると各成分の到着タイミングがずれ、時間波形のパルスは広がって隣のシンボルにはみ出す(符号間干渉、下図左)。ただし振幅を触らないので、**エネルギーも、信号に載った情報も失われない**。

N スパン進むと総分散は $\beta_2 \cdot (N \times 100 \text{ km})$ まで蓄積する。これに対し受信側で逆符号の位相

$$H_{\text{comp}}(\omega) = \exp\left(-j\frac{\beta_2}{2}\omega^2 z_{\text{total}}\right)$$

を掛けると、 $H_{CD} \cdot H_{\text{comp}} = 1$ となって位相回転がちょうど打ち消され、波形は **完全に元へ戻る** (下図右)。この操作が **デジタル分散補償** である。コヒーレント受信は光の複素振幅(振幅と位相の両方)をそのまま電気信号として取り出せるので、DSP 上で位相を逆向きに回すだけでよい。直接検波だと位相が失われてこの補償はできない。

2. なぜ BER が問4-5 と一致するのか

補償後の BER が損失のみの問4-5 とぴったり同じになる理由は、信号側と雑音側の両方を見ると分かる。

信号側は前節のとおりで、分散は可逆なので補償後は分散の無い場合と同じ QPSK 点に戻る。判定に使う信号点の位置は損失のみのときと変わらない。

雑音側がもう一つのポイントになる。EDFA が出す ASE 雑音は全周波数に均一に広がった白色雑音である。これが分散補償フィルタ(オールパス)を通っても、各周波数成分の振幅は変わらず位相だけが回る。白色雑音の電力は周波数ごとの振幅で決まるので、位相をいくら回しても電力も白色性も変化しない。各スパンの ASE が途中で何回分散を受けようと、最後に補償フィルタを通そうと、受け取る変換はすべてオールパスであり、受信端での雑音電力は損失のみの場合と同じ $N \cdot S_{\text{ASE}} \cdot R_s$ のままになる。

信号点が同じで雑音電力も同じなら、SNR も BER も問4-5 と一致する。

$$\text{SNR}_N = \frac{P_{\text{in}}}{N S_{\text{ASE}} R_s}, \quad \text{BER} = Q\left(\sqrt{\text{SNR}_N}\right)$$

受信感度($\text{BER} = 10^{-3}$ を満たす入力パワー)も問4-5 とまったく同じ値($-13.0, -10.1, \dots, +4.0$ dBm)になる。スパンが2倍になると雑音電力が2倍(+3 dB)になり、同じ SNR を保つには入力を約 3 dB 上げる必要がある。出力の各スパンが 3 dB 刻みで右へ寄っているのはこのためである。波長分散は線形である限り、補償すれば無害というのがこの問題の結論になる。

3. 数値実装の注意

このシミュレーションは 1 sample/symbol(1 シンボルを 1 サンプルで表す)で計算している。サンプリング間隔は $\Delta t = 1/R_s$ で、これを ps に換算してから β_2 [ps²/km] と単位をそろえる。分散はこのサンプル列のスペクトルに対するオールパス位相フィルタとして適用する。各スパンでは

分散 → 損失 → EDFA(+ASE)

の順に処理し、全スパンを通した後に総分散の逆を 1 回掛ける。分散ステップも補償ステップも線形なので、途中の ASE がどの順序で分散を受けても、トータルで見ればオールパス変換しか受けず、雑音電力は保存される。

実機では信号のスペクトルが帯域 $(1 + \alpha)R_s$ (ロールオフ α を含む)まで広がるため、それを取りこぼさないよう 2 sample/symbol 以上で処理する。サンプル数を増やしても、線形なら補償が完全という結論は変わらない。

4. 関数ごとの詳細

`solution.py` は補助関数 `dbm_to_w` と本体 `main` から成り、伝搬の部品は `_common/fiber.py`、QPSK の変復調は `_common/comm.py` を呼び出す。順に見ていく。

`dbm_to_w(dbm)`

入力パワーの単位を dBm からワットに直す。

項目	内容
引数	<code>dbm</code> : パワー dBm 。
戻り値	パワー [W]。

中身は `10 ** (dbm / 10.0) * 1e-3` の 1 行。dBm は 1 mW を基準にしたデシベル表記なので、`10^(dBm/10)` で mW に直し、`1e-3` を掛けて W にする。例えば 0 dBm は 1 mW = 0.001 W になる。

`main()`

全体をつなぐ関数。準備、伝搬ループ、作図の 3 段に分かれる。

準備の段。

- 乱数生成器 `rng` を固定シード `SEED=22` で作る。ASE 雑音の再現性のため。
- スパン利得 `G = 10^(0.3*100/10)` (= 30 dB)を求める。損失をちょうど打ち消す利得。
- `fiber.alpha_from_db` で損失係数 [dB/km] を振幅減衰係数 α [1/km] に、`fiber.ase_psd` で 1 スパンあたりの ASE パワースペクトル密度 S_{ASE} を計算する。
- 角周波数グリッド `omega` を作る。`dt_ps = 1e12 / RS` でサンプル間隔を ps にし、`np.fft.fftfreq` から $\omega = 2\pi f$ [rad/ps] を得る。これが分散ステップで使う ω になる。
- `comm.generate_prbs(15)` で PRBS を作り、`comm.bits_to_symbols(..., 4)` で QPSK(M=4)シンボルに マップする。それを `N_SYM=100000` シンボル分タイルで並べ、送信ビット `bits_tx` を控えておく。

伝搬ループの段。外側がスパン数 `SPANS`、内側が入力パワー `pin_dbm` (−16〜+12 dBm)の二重ループ。内側 1 回で次をやる。

- `Pin = dbm_to_w(pdbm)` をワットに直し、`A = sqrt(Pin) * s` で送信振幅を作る ($|A|^2$ が光パワー)。
- `nspan` 回ループし、各スパンで `fiber.dispersion_step` (分散)→ `fiber.loss_step` (損失)→ `fiber.edfa` (増幅と ASE 付加)を順に適用する。
- 補償前の信号 `A_disp = A.copy()` を控える(後でコンステレーション用)。
- `fiber.dispersion_step(A, BETA2, -nspan*L_SPAN_KM, omega)` で総分散を逆向きに 1 回掛けて補償する。距離が `-nspan*L_SPAN_KM` と負なので、行きと逆の位相になる。

5. `r = A / sqrt(Pin)` で振幅スケールを戻し、`comm.symbols_to_bits(r, 4)` で最近傍判定してビットに戻す。
6. `comm.count_bit_errors` で送受ビットを比べ、誤り数をビット数で割って BER とする。
7. 理論値は `snr_lin = Pin / (nspan * s_ase * RS)` を `comm.ber_theory_qam` に渡して計算する。
8. 10 スパン・ $P_{in} = 2$ dBm のときだけ、補償前後の信号を `cap` に保存しておく。

掃引が終わったら、各スパンについて BER が 10^{-3} を横切る入力パワーを `np.interp` で補間して受信感度を求め、表示する。`r = A / sqrt(Pin)` で割って戻している 1 行が効いていて、信号点を理想振幅 1 に正規化することで、雑音の相対的な大きさ(= SNR)だけが BER を決める形になる。

作図の段。

1. BER 図(`multispan_cd_ber.png`): スパンごとに理論曲線(実線)とシミュレーション点(丸)を片対数で重ねる。色はスパン数で変える。すべての曲線が問4-5 と重なることを示す。
2. コンステレーション図(`cd_compensation.png`): 控えておいた補償前 `cap["before"]` と補償後 `cap["after"]` を左右に並べ、分散で崩れた QPSK が補償でききれいに戻る様子を対比する。

呼び出している `fiber.py` の部品

関数	役割	本問での使い方
<code>dispersion_step(A, β_2, z, ω)</code>	距離 z の分散をスペクトルに掛ける	順方向は <code>z=L_span</code> 、補償は <code>z=-N*L_span</code>
<code>loss_step(A, α, z)</code>	振幅を <code>$\exp(-\alpha/2 \cdot z)$</code> 倍に減衰	各スパン 100 km の損失
<code>edfa(A, G, NF, fs, rng)</code>	振幅を <code>$\sqrt{G}$</code> 倍し ASE 雑音を足す	各スパン末尾で損失を補償
<code>ase_psd(G, NF)</code>	ASE のパワースペクトル密度 S_{ASE}	理論 SNR の分母に使う
<code>alpha_from_db(dB/km)</code>	損失 [dB/km] を α [1/km] に換算	<code>loss_step</code> に渡す α

`dispersion_step` の中身は `ifft(fft(A) * exp(j*( $\beta_2/2$ )* $\omega^2 \cdot z$ ))` で、まさに第 1 節のオールパス位相フィルタを スペクトルに掛けて時間波形に戻している。補償が距離を負にするだけで済むのは、この位相が z に比例する 線形量だからである。

5. 実行

`python solution.py`

標準出力は冒頭の「実行結果(出力)」、生成図は `multispan_cd_ber.png` と `cd_compensation.png` を参照。
処理フロー図 `flow.png` は `python make_flow.py` で作り直せる。

6. この問題のつながり

ここまでの問4-5・問4-6 では、伝送路の要素(損失・分散・ASE)がすべて **線形**だった。線形な範囲では P_{in} を上げるほど SNR が上がり、BER は下がる一方になる。分散のような線形ひずみは、補償すればペナルティを残さずに取り除ける。

最後の問4-7 では **非線形効果**(自己位相変調 SPM)を加える。非線形ひずみは入力パワーが大きいほど強くなり、分散補償では取り除けない。その結果、 P_{in} を上げすぎると今度は BER が悪化に転じ、ちょうどよい **最適入力パワー**が存在するようになる。